

Отчет по Лабораторной работе №3:

Проектирование NoSQL базы данных

1. Проектирование NoSQL БД

Для системы напоминаний о приеме препаратов была выбрана архитектура с использованием двух типов БД (Polyglot Persistence): реляционной (PostgreSQL) для хранения структурированных данных и NoSQL (ClickHouse) для хранения огромных массивов логов и проведения аналитики.

Описание хранимых данных (ClickHouse)

В NoSQL БД переносятся данные о фактических приемах лекарств, так как этот объем данных растет наиболее быстро и требует высокой скорости агрегации.

Сущность: `intake_logs` (Журнал приемов)

Атрибут	Тип данных	Описание	Ограничения
<code>log_id</code>	UInt64	Уникальный идентификатор записи	PK (в составе Order By)
<code>patient_id</code>	UUID	ID пациента (связь с PostgreSQL)	Индексировано
<code>drug_id</code>	UInt32	ID препарата (связь с PostgreSQL)	-
<code>schedule_id</code>	UInt32	ID правила приема (связь с PostgreSQL)	-
<code>actual_time</code>	DateTime	Точное время фиксации приема	Индексировано
<code>status</code>	Enum8	Статус приема ('Taken'=1, 'Skipped'=2)	-
<code>dosage</code>	String	Дозировка препарата в момент приема	-

Связи: Данные в ClickHouse хранятся в денормализованном виде для обеспечения максимальной скорости чтения. Связь с реляционной БД осуществляется через `patient_id`, `drug_id` и `schedule_id`.

2. Перечень основных операций с NoSQL БД

На основе функциональных требований системы определены следующие операции:

- Добавление данных:** Запись каждого факта приема или пропуска препарата в режиме реального времени (`INSERT`).
- Аналитический запрос (Приверженность):** Расчет процента пропущенных приемов по конкретному пациенту за период (Агрегация `COUNT` с фильтром по `status`).

3. **Аналитический запрос (Популярность):** Определение препаратов, которые чаще всего пропускаются пациентами (Группировка по `drug_id`).
4. **Аналитический запрос (Тренды):** Построение графика количества приемов по дням для мониторинга нагрузки на систему.
5. **Удаление данных:** Очистка старых логов (например, старше 5 лет) с помощью `ALTER TABLE ... DROP PARTITION` .

3. Выбор СУБД

Выбранная СУБД: Apache ClickHouse. **Обоснование:** ClickHouse является колоночной СУБД, что делает её идеальной для аналитических задач (OLAP). В данной системе лог приемов будет расти экспоненциально. ClickHouse обеспечивает:

- Высокую скорость сжатия данных.
- Молниеносное выполнение агрегатных функций (SUM, AVG, COUNT) по миллионам строк.
- Эффективную работу с временными рядами (Time Series), что критично для медицинских журналов.

4. Установка и настройка

СУБД развернута с использованием Docker Compose.

- **Образ:** `clickhouse/clickhouse-server:latest`
- **Порты:** `8123` (HTTP интерфейс для DBEaver), `9000` (Native protocol).
- **Средство работы:** DBEaver (подключение через JDBC драйвер ClickHouse).

5. Создание базы данных

База данных была создана с помощью SQL-скрипта `lab3/init_clickhouse.sql` . Использован движок `mergeTree` с партиционированием по месяцам (`toYYYYMM(actual_time)`) и сортировкой по (`patient_id, actual_time`) .

6. Наполнение данными

База данных наполнена тестовыми данными с помощью скрипта `lab3/seed_clickhouse.sql` (15 записей, охватывающих разных пациентов и разные статусы приема).

Проверка данных:

```
SELECT * FROM intake_logs LIMIT 10;
```

(Скриншот результата прилагается к отчету в виде отдельного приложения)